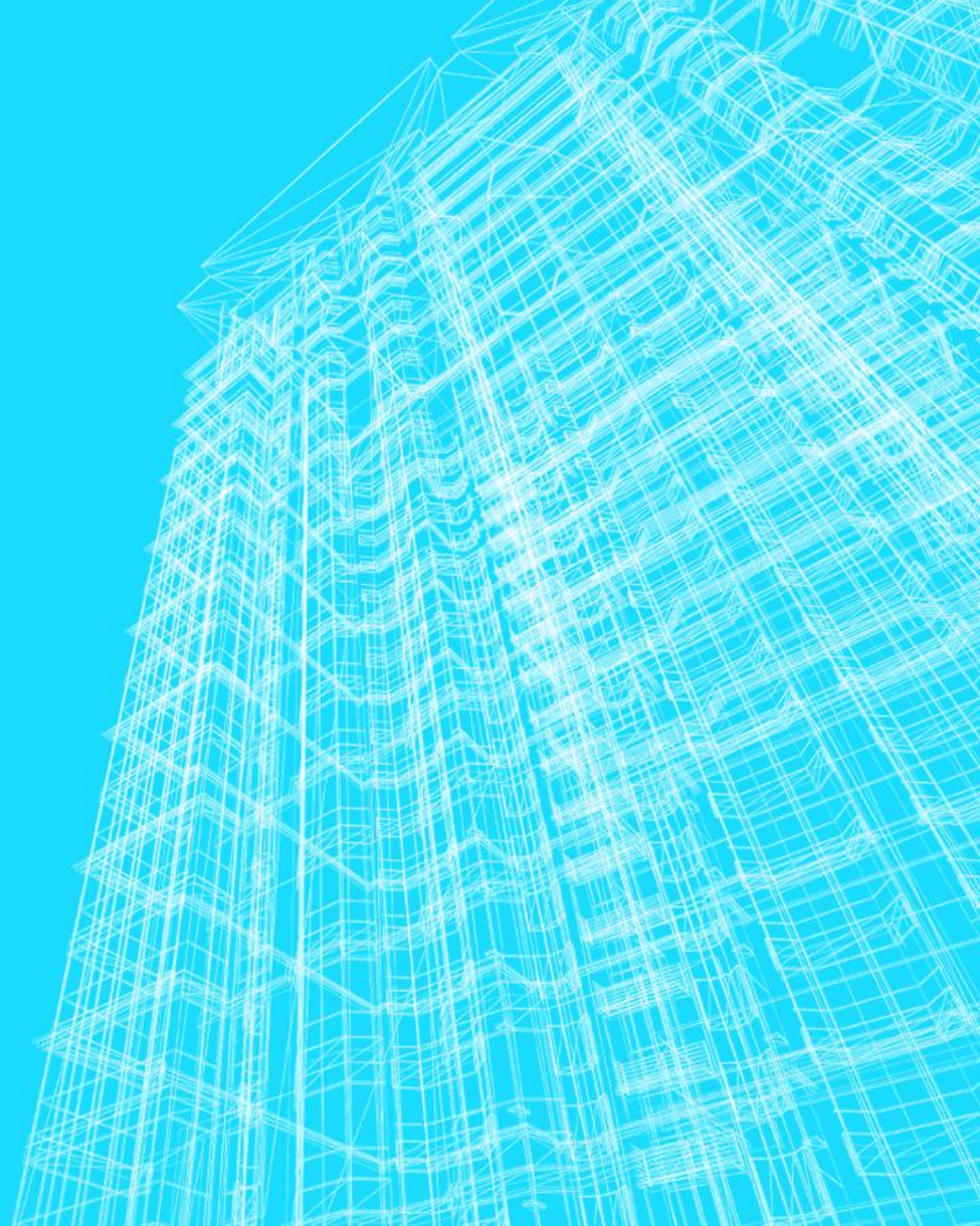


# ETL PROCES

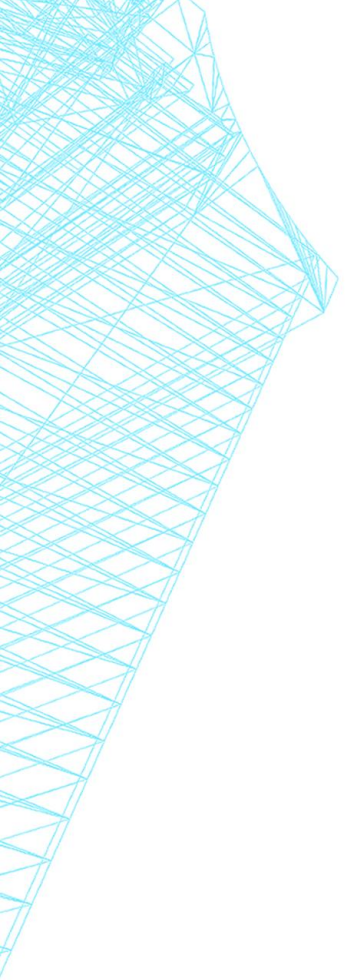
VJEŽBE 4



# SADRŽAJ VJEŽBI

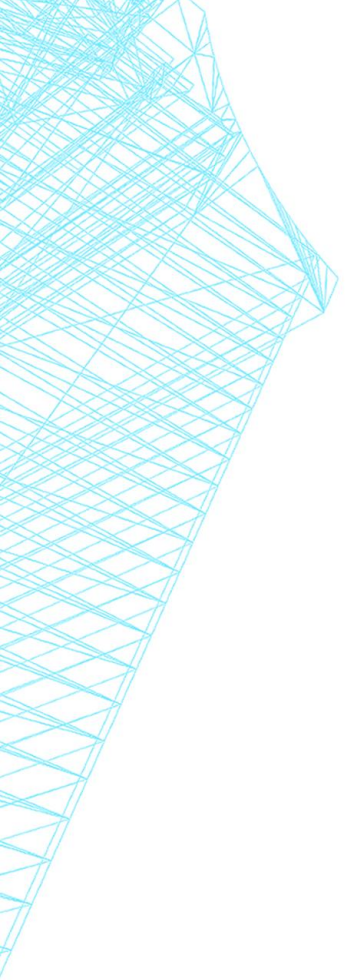
- ETL proces
  - izdvajanje podataka
  - transformacija
  - punjenje podatka

```
✓ FIPU_SRP_VJEZBE
  ✓ Vjezbe4_ETL
    ✓ extract
      ✚ extract_mysql.py
    ✓ load
      > __pycache__
      ✚ run_loading.py
    ✓ transform
      > __pycache__
    ✓ dimensions
      > __pycache__
      ✚ country_dim.py
      ✚ date_dim.py
      ✚ product_dim.py
      ✚ retailer_dim.py
    ✓ facts
      > __pycache__
      ✚ sales_fact.py
      ✚ pipeline.py
      ✚ etl_pipeline.md
      ✚ main.py
      ✚ provjera.sql
      ✚ spark_session.py
```



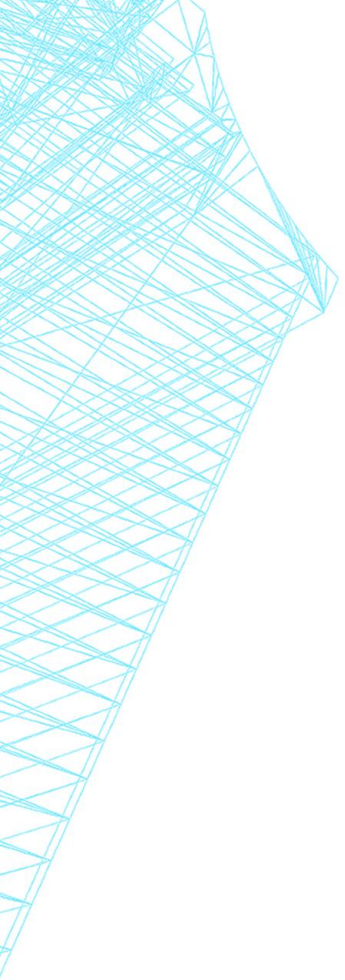
# PRIPREMA I MODELIRANJE

- izrada modela skladišta podataka
- entiteti modela
  - tablice činjenica
  - dimenzijske tablice
- relacije
- atributi
  - surogat ključevi
  - hijerarhije



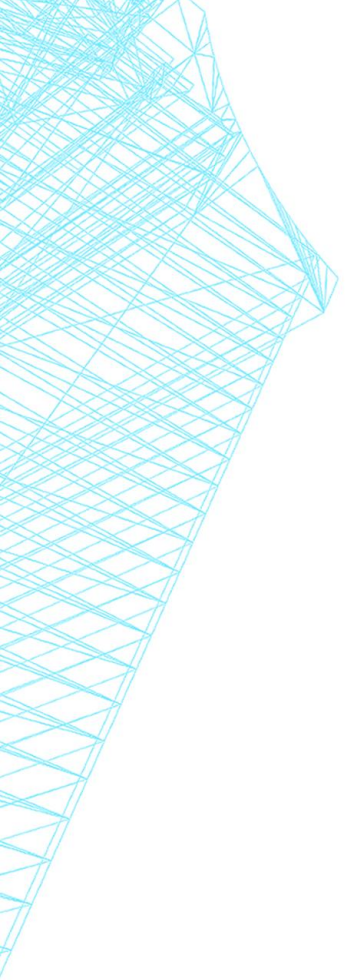
# PRIPREMA I MODELIRANJE

- napredne teme
- vrste dimenzija
  - mini dimenzije
  - degenerirane dimenzije
  - usklađene dimenzije
  - sporo mijenjajuće dimenzije
- vrste tablica činjenica
  - modeli
  - pravovremenost podataka



# PRIPREMA I MODELIRANJE

- jednom stvoren dimenzijski model podataka potrebno je puniti podacima iz transakcijskog sustava
- složen proces s mnogo parametara
  - način punjenja, praćenje promjena. frekvencija punjenja
- osnovne operacije u procesu punjenja su:
  - izdvajanje
  - transformiranje
  - učitavanje



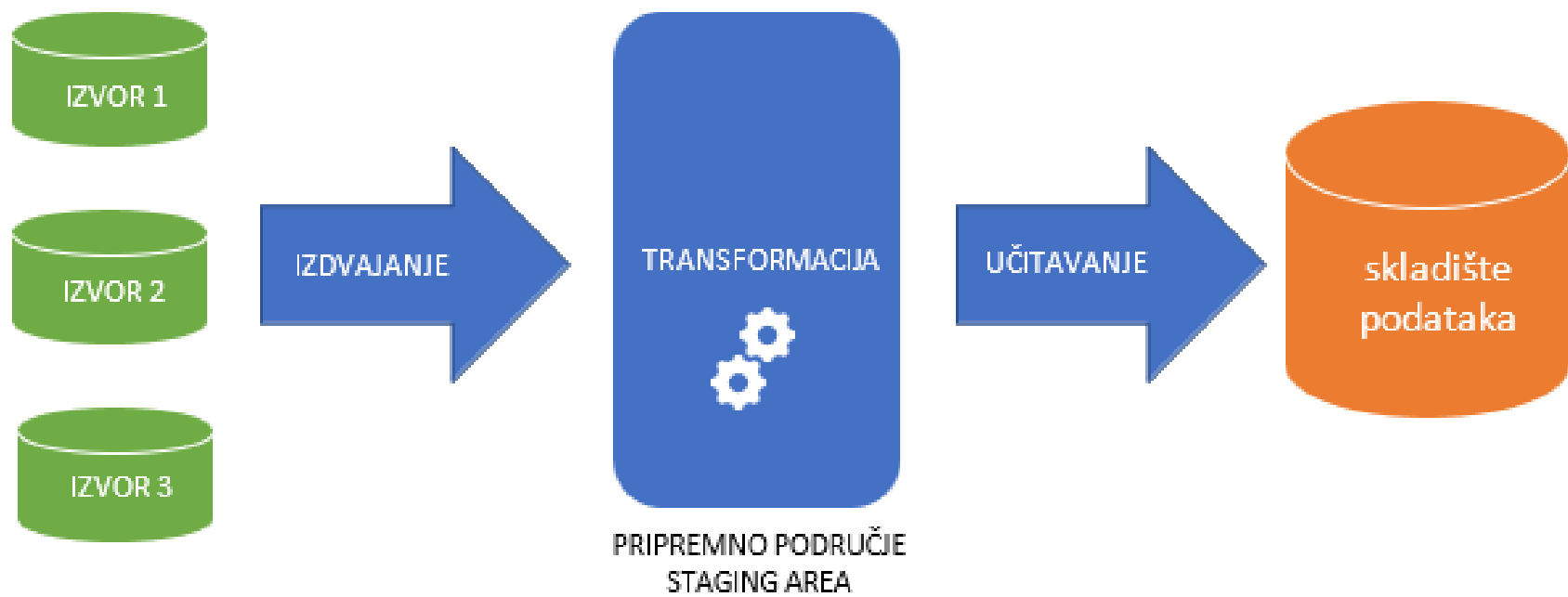
# ETL PROCES

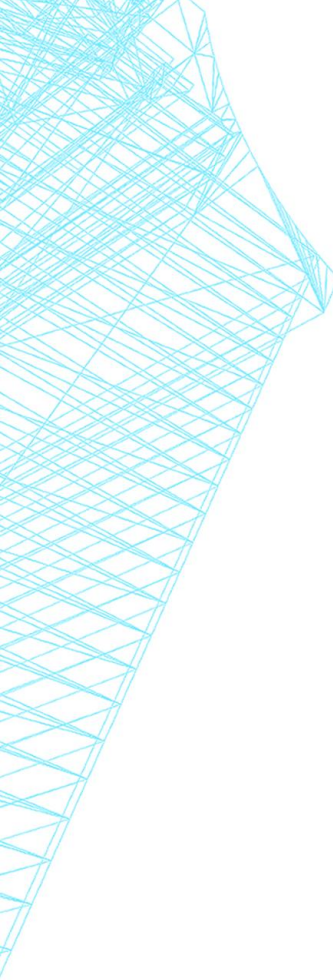
- ETL proces (Extract, Transform, Load)
- odgovoran je za:
  - izdvajanje podataka iz transakcijskog sustava u pripremno područje
  - transformaciju podataka
    - čišćenje podataka
    - transformacija u strukturu za dimenzijski model
  - učitavanje podataka u skladište podataka
- zahtjevan dio izrade skladište podataka (implementacija i održavanje)
- specifičan za svaku organizaciju, sustav

# ETL PROCES

- svaki korak ima svoje izazove:
- *izdvajanje (ekstrakcija) podataka*
  - detektiranje promjena
  - više izvora podataka...
- *transformacija i čišćenje*
  - način transformacije i čišćenja
  - mogućnost različitosti podataka...
- *punjenje skladišta podataka*
  - način učitavanja...

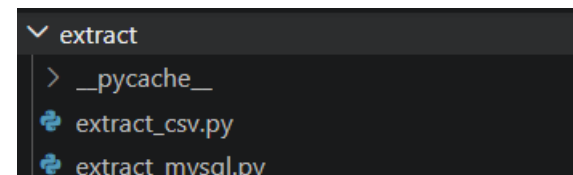
```
✓ Vjezbe4_ETL
  > __pycache__
  ✓ Connectors
    ■ mysql-connector-j-9.2.0.jar
  ✓ extract
    > __pycache__
    ■ extract_csv.py
    ■ extract_mysql.py
  ✓ load
    > __pycache__
    ■ run_loading.py
  ✓ transform
    > __pycache__
    > dimensions
    > facts
    ■ pipeline.py
  ↓ etl_pipeline.md
  ■ main.py
  ■ provjera.sql
  ■ spark_session.py
```

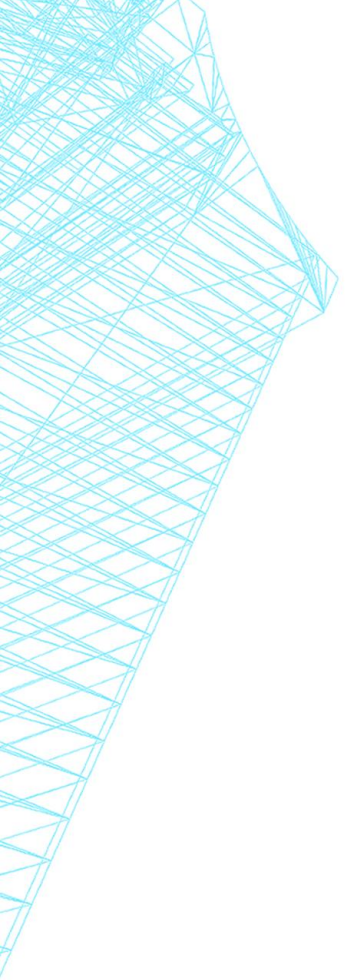




# IZDVAJANJE PODATAKA

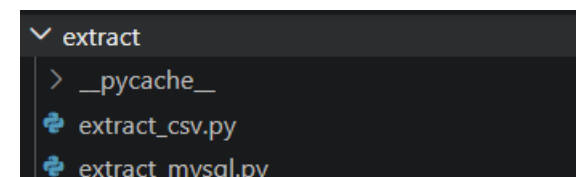
- bitan aspekt izdvajanja podataka je briga o transakcijskom
- (izvorišnom) sustavu organizacije
- produkcijski sustav ne smije biti ugrožen prikupljanjem podataka za skladište podataka
- potrebno je paziti:
  - da se podaci ne mijenjaju (ili izgube) u izvorišnom sustavu
  - da se ne (ili što manje) mijenja konfiguracija sustava
  - na performanse (zaključavanje) izvorišnog sustava prilikom izdvajanja
- podataka (vrijeme izdvajanja podataka?)

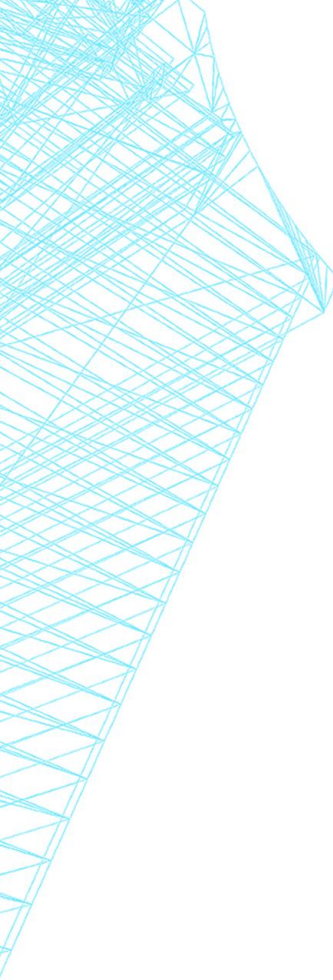




# IZDVAJANJE PODATAKA

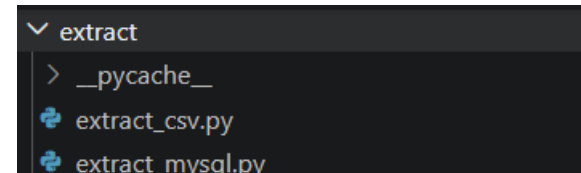
- Pripremno područje - *engl. staging area*
- pripremno područje izvornih podatka za transformaciju i čišćenje
- podataka
  - kopija transakcijskog sustava
- korisnici nemaju pristup pripremnom području
- upiti ne mogu pristupiti podacima u pripremnom području
  - tablice mogu biti dodane ili izbrisane bez pravila
- iz pripremnog područja se puni skladište podataka





# IZDVAJANJE PODATAKA

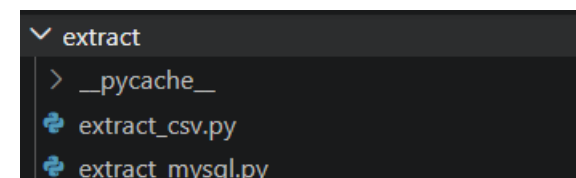
- način izdvajanja podataka:
- potpuno
  - prvo izdvajanje (*initial load*)
  - *full load*
- djelomično
  - inkrementalna nadopuna podacima koji su se promijenili od vremena zadnjeg
- uspješnog ETL procesa
  - različite strategije za pronalazak promjena u izvorišnom sustavu

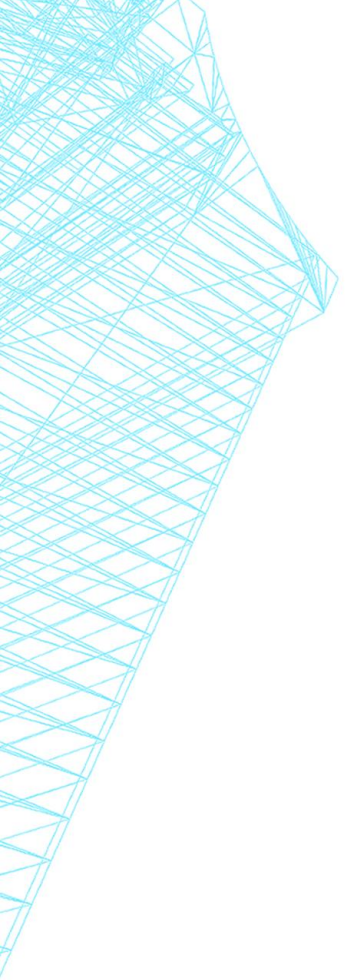


```
▼ extract
> __pycache__
extract_csv.py
extract_mysql.py
```

# IZDVAJANJE PODATAKA

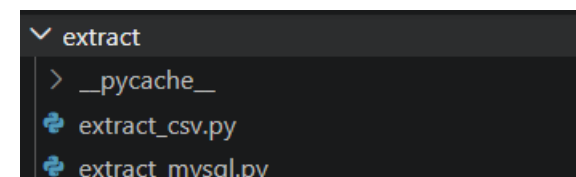
- izdvajanje podataka se odvija periodično (ovisno o potrebama tvrtke),
- pozadinskim procesom
- **prvo punjenje** podacima prilikom uspostave skladišta podataka
  - obuhvaća sve dostupne podatke iz izvora
- često je manje komplicirano od periodičkog osvježavanja
  - manje problema
  - ne mora se voditi briga već o učitanim podacima
- zahtjeva posebnu verziju ETL procesa

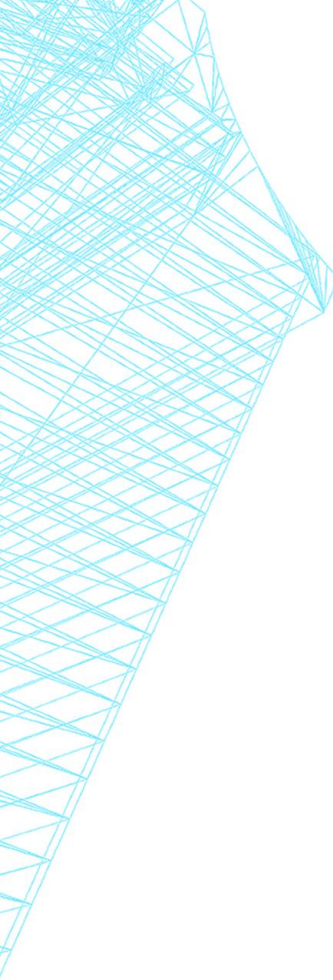




# IZDVAJANJE PODATAKA

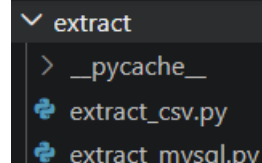
- dodavanje vremenske oznake
- dodavanje posebnog atributa koji prati vrijeme
  - dan, sat, minuta, sekunda
- ETL proces uspoređuje vrijeme zadnje ekstrakcije i oznaku vremena
- na pojedinom zapisu
- ograničenja:
  - zadnja izmjena
  - obrisani zapisi (A)



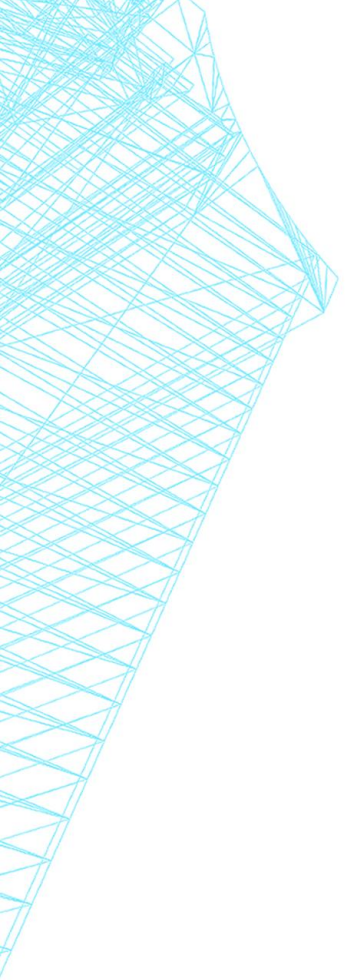


# IZDVAJANJE PODATAKA

- aplikacijsko bilježenje promjena
- bilježenje promjena na posebno mjesto (tablicu) predviđeno za tu namjenu
- svi aplikacijski programi moraju jednako evidentirati promjene da bi sustav funkcionirao
- složeno za implementaciju, potreban je veliki angažman za promjenu aplikacija
- pogodno ukoliko se koriste baze podataka koje ne podržavaju mehanizme za praćenje promjena



```
▼ extract
  > __pycache__
  extract_csv.py
  extract_mysql.py
```

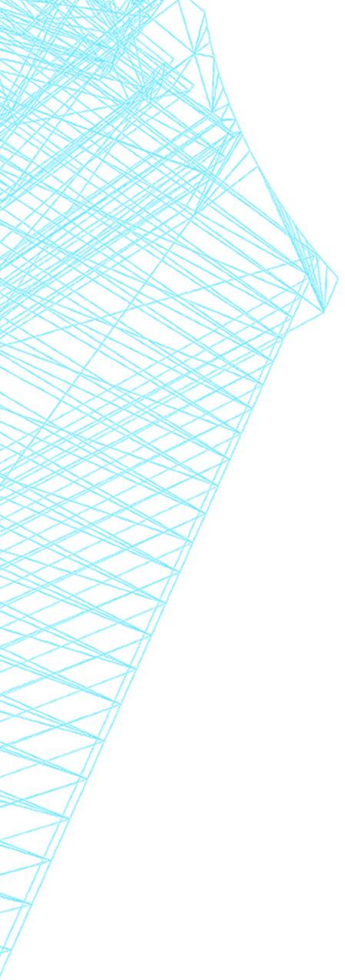


```
Vjezbe4_ETL > extract > extract_csv.py > ...
```

```
1 # extract/extract_csv.py
2 from spark_session import get_spark_session
3
4 def extract_from_csv(file_path):
5     spark = get_spark_session("ETL Extract - CSV")
6     df = spark.read.option("header", True).option("inferSchema", True).csv(file_path)
7     return df
8
```

```
Vjezbe4_ETL > extract > extract_mysql.py > ...
```

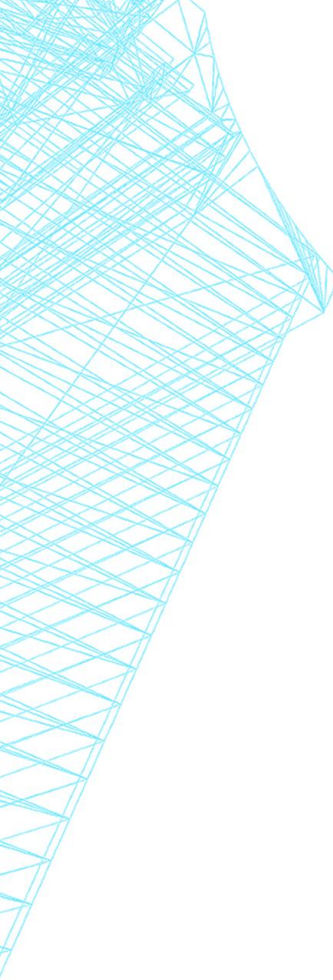
```
1 # extract/extract_mysql.py
2 from spark_session import get_spark_session
3
4 def extract_table(table_name):
5     spark = get_spark_session("ETL App")
6
7     jdbc_url = "jdbc:mysql://127.0.0.1:3306/dw?useSSL=false"
8     connection_properties = {
9         "user": "root",
10        "password": "1234",
11        "driver": "com.mysql.cj.jdbc.Driver"
12    }
13
14    df = spark.read.jdbc(url=jdbc_url, table=table_name, properties=connection_properties)
15    return df
16
17 def extract_all_tables():
18     return {
19         "product": extract_table("product"),
20         "product_type": extract_table("product_type"),
21         "product_line": extract_table("product_line"),
22         "sales": extract_table("sales"),
23         "order_method": extract_table("order_method"),
24         "retailer_type": extract_table("retailer_type"),
25         "country": extract_table("country"),
26     }
```



# TRANSFORMACIJA PODATAKA

- drugi korak u ETL procesu
- izvodi se u pripremnom području
  - razlozi stvaranja pripremnog područja?
- transformacija podrazumijeva:
  - analizu podataka
  - čišćenje podataka
  - transformaciju u odgovarajuću strukturu za dimenzijski model podataka
- operacije se izvršavaju slijednim redoslijedom (*pipelining order*)

```
▼ transform
  > __pycache__
  ▼ dimensions
    > __pycache__
    🔄 country_dim.py
    🔄 date_dim.py
    🔄 product_dim.py
    🔄 retailer_dim.py
  ▼ facts
    > __pycache__
    🔄 sales_fact.py
    🔄 pipeline.py
```



# TRANSFORMACIJA PODATAKA

- čišćenje podataka – zašto?
- kvaliteta podataka koji se učitavaju u skladište podatka!
- bitne karakteristike:
  - ispravnost
  - nedvosmislenost
  - konzistentnost
  - potpunost

```
▼ transform
  > __pycache__
  ▼ dimensions
    > __pycache__
    🔄 country_dim.py
    🔄 date_dim.py
    🔄 product_dim.py
    🔄 retailer_dim.py
  ▼ facts
    > __pycache__
    🔄 sales_fact.py
    🔄 pipeline.py
```

# TRANSFORMACIJA PODATAKA

- testovi kvalitete (engl. Quality Screens QS)
- su testovi u procesu ETL-a, koji se javljaju ukoliko podaci ne prođu određeni test kvalitete
- ukoliko se javi greška aktivira se QS, i bilježi se u shemu grešaka (*engl. error event shema*)
- različite vrste testova grešaka:
  - stupčani testovi
  - strukturni testovi
  - testovi poslovnih pravila

```
▼ transform
  > __pycache__
  ▼ dimensions
    > __pycache__
    🔄 country_dim.py
    🔄 date_dim.py
    🔄 product_dim.py
    🔄 retailer_dim.py
  ▼ facts
    > __pycache__
    🔄 sales_fact.py
    🔄 pipeline.py
```

# TRANSFORMACIJA PODATAKA

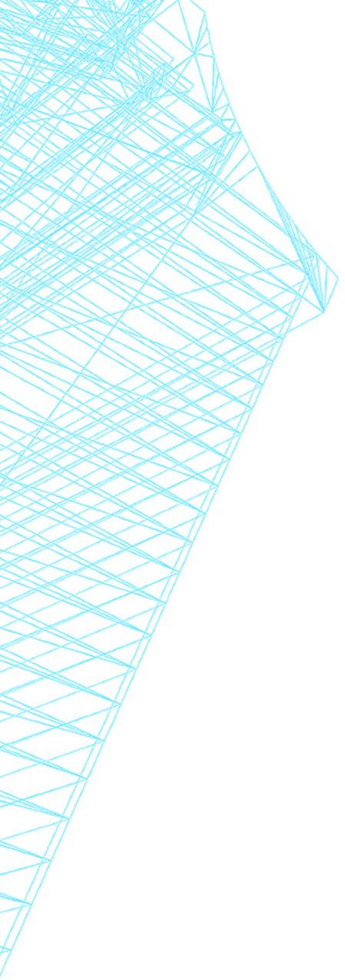
- shema grešaka
- dimenzijski modela koji ima cilj evidentiranja svih grešaka koje su se dogodile u ETL procesu
- ukoliko se aktivira test kvalitete bilježi se u shemu grešaka
- tablica činjenica je događaj greške
  - granularnost – svaka greška u ETL-u
- dimenzije
  - vremenska, obrada u kojoj se greška dogodila, test kvalitete...

```
▼ transform
  > __pycache__
  ▼ dimensions
    > __pycache__
    🔄 country_dim.py
    🔄 date_dim.py
    🔄 product_dim.py
    🔄 retailer_dim.py
  ▼ facts
    > __pycache__
    🔄 sales_fact.py
    🔄 pipeline.py
```

# TRANSFORMACIJA PODATAKA

- revizorska dimenzija (*engl. audit dimension*)
- činjeničnoj tablici se pridjeljuje dimenzija s metapodacima koji opisuju kvalitetu podataka
- pridjeljuju se svakom zapisu tablice činjenica
- sadrži ETL podatke o procesiranju i kvaliteti podataka
- može se koristiti shema grešaka da bi se ispitalo da li je bilo problema
- s kvalitetom tijekom ETL procesa

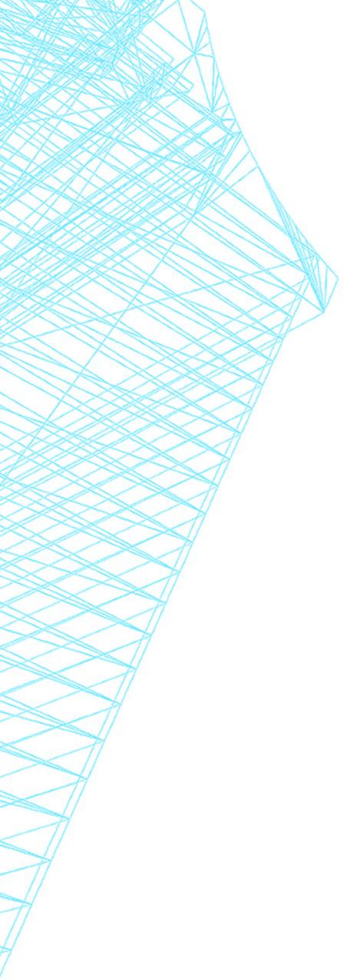
```
▼ transform
  > __pycache__
  ▼ dimensions
    > __pycache__
    🔄 country_dim.py
    🔄 date_dim.py
    🔄 product_dim.py
    🔄 retailer_dim.py
  ▼ facts
    > __pycache__
    🔄 sales_fact.py
    🔄 pipeline.py
```



# TRANSFORMACIJA PODATAKA

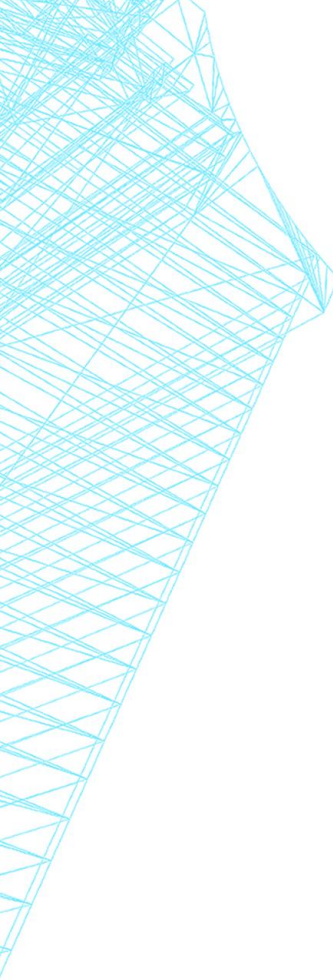
- transformacija u odgovarajuću strukturu za dimenzijski model
- podataka
- izgradnja dimenzija
  - uklanjanje duplih dimenzijskih zapisa
  - mehanizam sporo mijenjajućih dimenzija
  - generiranje surogat ključeva
- izgradnja tablice činjenica
  - povezivanje s dimenzijama

```
▼ transform
  > __pycache__
  ▼ dimensions
    > __pycache__
    🔄 country_dim.py
    🔄 date_dim.py
    🔄 product_dim.py
    🔄 retailer_dim.py
  ▼ facts
    > __pycache__
    🔄 sales_fact.py
    🔄 pipeline.py
```



# PUNJENJE PODATAKA

- nakon transformacije podataka dolazi punjenje u dimenzijski model podataka
- punjenje podataka je moguće:
  - odjednom
  - zapis po zapis
- odabir metoda ovisi o specifičnom procesu, punjenje odjednom je
- vremenski manje zahtjevno
- akcije prilikom punjenja podataka, zapise treba:
  - dodati
  - izmijeniti
  - brisati



```
from pyspark.sql import DataFrame
```

```
def write_spark_df_to_mysql(spark_df: DataFrame, table_name: str, mode: str = "overwrite"):
```

```
    # URL tvoje baze - provjeri da se baza stvarno zove 'dw'
```

```
    jdbc_url = "jdbc:mysql://127.0.0.1:3306/dw?useSSL=false"
```

```
    connection_properties = {
```

```
        "user": "root",
```

```
        "password": "1234", # Tvoja lozinka za MySQL
```

```
        "driver": "com.mysql.cj.jdbc.Driver"
```

```
    }
```

```
    print(f"🚀 Writing to table `{table_name}` with mode `{mode}`...")
```

```
    try:
```

```
        spark_df.write.jdbc(
```

```
            url=jdbc_url,
```

```
            table=table_name,
```

```
            mode=mode,
```

```
            properties=connection_properties
```

```
        )
```

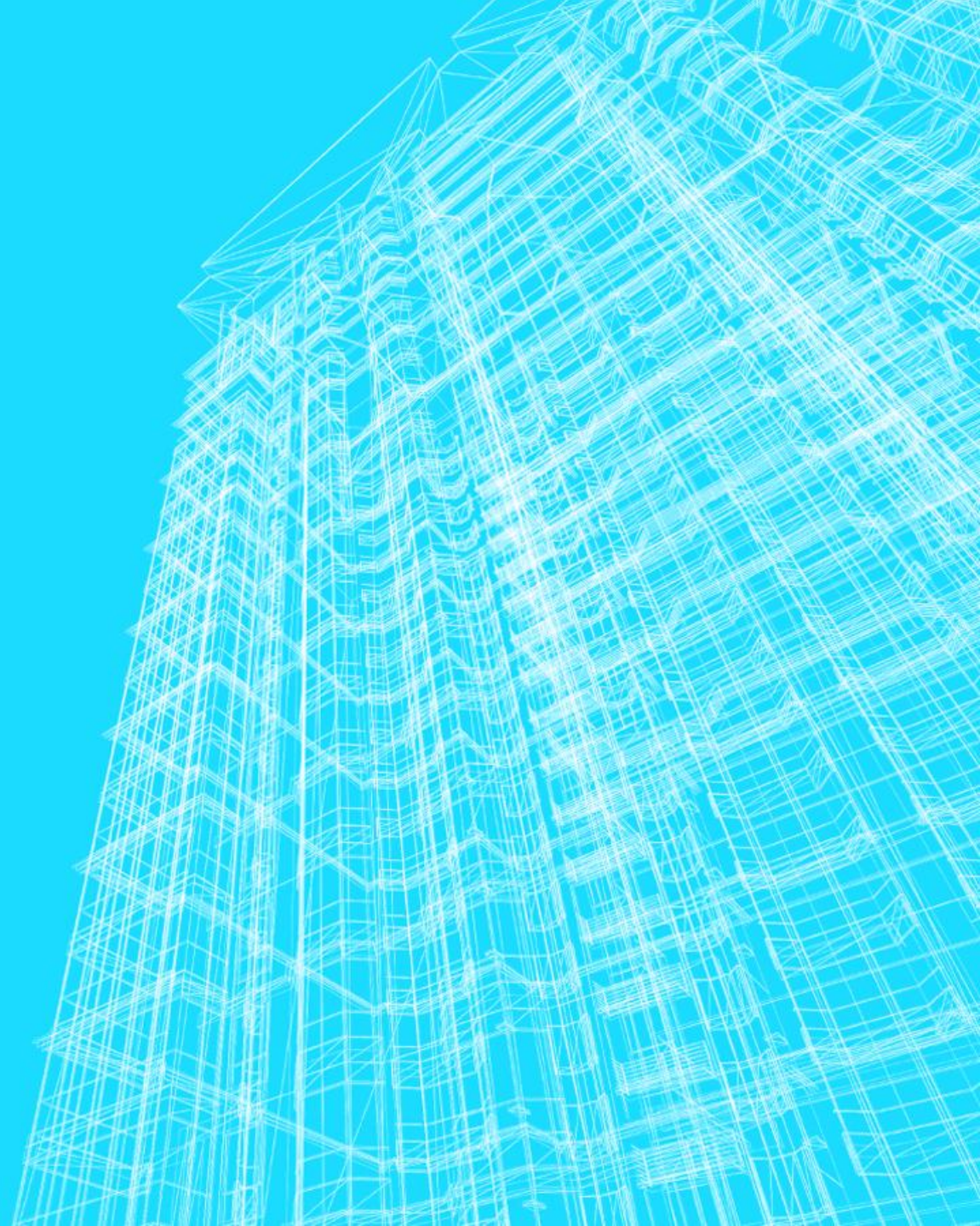
```
        print(f"✅ Successfully written to `{table_name}`.")
```

```
    except Exception as e:
```

```
        print(f"❌ Error writing to `{table_name}`: {e}")
```

# PRIMJER

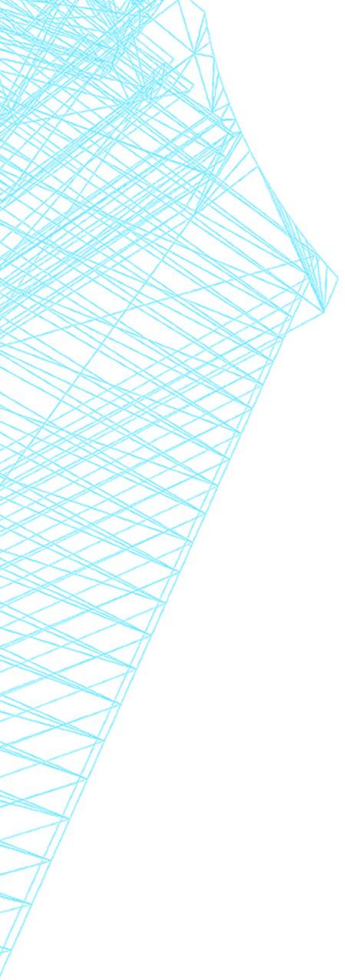
OPREMA D.D.



# TIPS AND TRICKS

🔧 Pre-requisites: Što sve treba?

- Prije nego što uopće dotaknete kod, morate instalirati ove komponente (inače ništa neće raditi):
  - **Python biblioteke:** pip install pandas sqlalchemy pymysql pyspark
  - **Java (za Spark):** Spark ne radi bez Jave. Morate imati instaliran **JDK 17**.
  - **MySQL JDBC Driver:** .jar datoteka (npr. mysql-connector-j-8.x.jar) koja mora biti u Sparkovom jars folderu ili specificirana u kodu.
  - **Hadoop Winutils (za Windows):** Ako ste na Windowsima, trebaju winutils.exe u C:\hadoop\bin mapi, inače će Spark javljati grešku pri pisanju podataka.

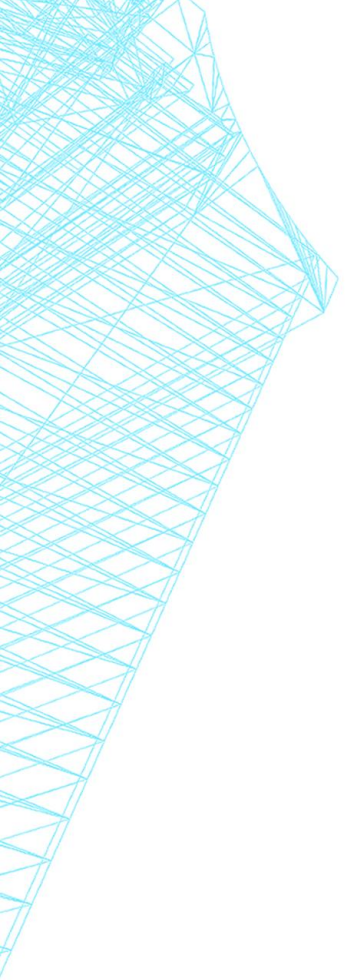


# TIPS AND TRICKS

## POTENCIJALNI PROBLEMI

### 1. Problem: "Ne mogu obrisati tablicu zbog stranih ključeva!"

- Foreign keys štite bazu, ali nam smetaju kod resetiranja ETL-a.
- Uvijek gasite provjeru ključeva prije brisanja, ili brišite tablice obrnutim redoslijedom: prvo fact, pa dim.“
- **SQL kod:** SET FOREIGN\_KEY\_CHECKS = 0;

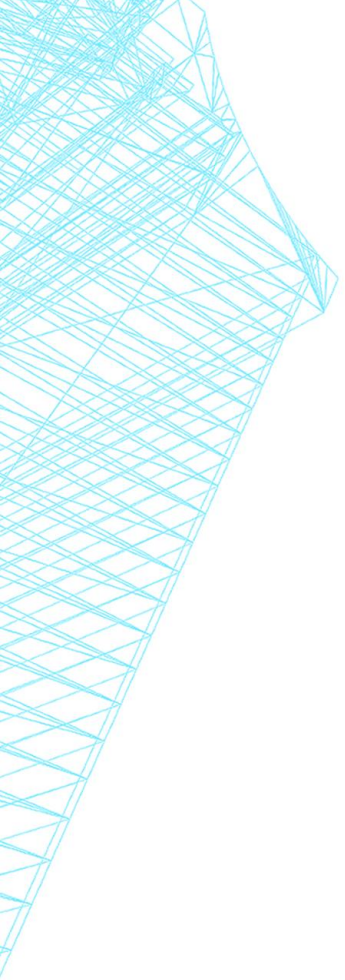


# TIPS AND TRICKS

## POTENCIJALNI PROBLEMI

### 2. Problem: "Unknown Column: c.nesto\_nesto"

- Spark i MySQL se ponekad ne "dogovore" oko velikih i malih slova ili točnih naziva.
- Nikad ne pretpostavljajte kako se stupac zove.
- Uvijek prvo pokrenite DESCRIBE dim\_nesto; u Workbenchu da vidite točan naziv.
- **Zlatno pravilo:** Koristite mala slova i donje crte (\_) za sve nazive u bazi.

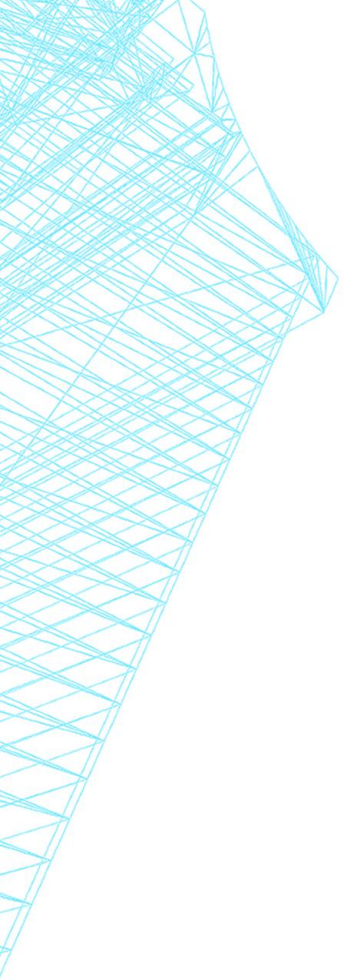


# TIPS AND TRICKS

## POTENCIJALNI PROBLEMI

### 3. Problem: "Prazan rezultat nakon JOIN-a"

- Ako spajaš Integer na String, rezultat je nula, a nema greške.
- Provjerite tipove podataka.
- Ako je nesto\_id u jednoj tablici tekst, a u drugoj broj, JOIN će biti prazan.
- Koristite `.cast("int")` u Sparku da ih izjednačite.



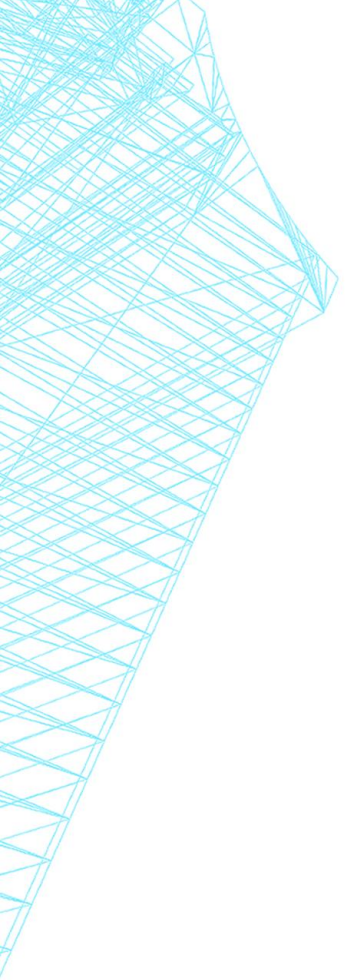
# TIPS AND TRICKS

## POTENCIJALNI PROBLEMI

### 4. Problem: "Spark kuka o Hadoop putanji"

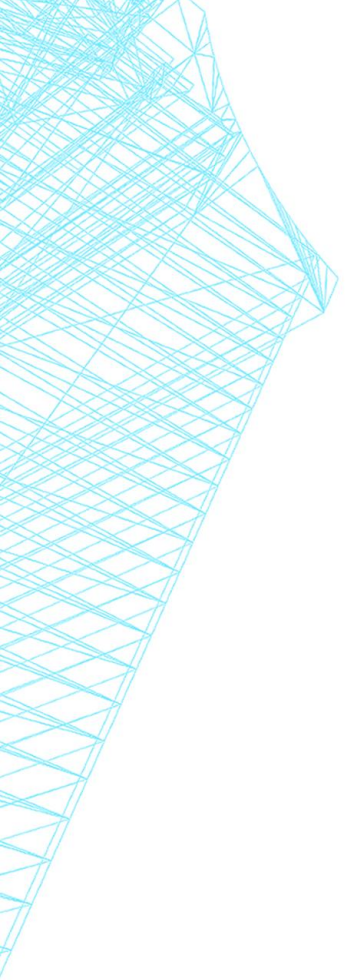
- Spark na Windowsima traži HADOOP\_HOME.

```
import os  
os.environ['HADOOP_HOME'] = "C:/hadoop"
```



# TIPS AND TRICKS - DODATAK

- **Idempotentnost:** ETL skripta mora biti takva da se može pokrenuti 10 puta zaredom bez da se podaci uduplaju. (Zato koristimo overwrite ili TRUNCATE).
- **Surogatni vs. Prirodni ključevi:**
  - **Prirodni:** ID koji dobijemo izvana (npr. šifra proizvoda).
  - **Surogatni (\_tk):** Naš interni broj (1, 2, 3...).
  - **Zašto?** Jer ako se vanjska šifra promijeni, mi ne želimo da nam se sruši cijela povijest prodaje u skladištu.
- **SCD Type 2:** stupci date\_from i date\_to. To je "vremenski stroj" baze podataka. Bez toga ne znamo tko je gdje živio ili koliko je proizvod koštao prošle godine.
- **Redoslijed punjenja:**
  - Prvo punimo sve dim\_ tablice.
  - Uzimamo njihove nove \_tk ključeve.
  - Tek onda punimo fact\_nesto.



# CHECKPOINT 4

- napuniti skladište podataka iz barem 2 izvora
- **provesti ETL proces**
- provjeriti unos podataka
- provjeriti da nema izgubljenih podataka

## OBAVEZNO!

- Tableau licenca (edu) – ukoliko planirate raditi vizualizaciju u njemu.